

PANDAS

INTRODUCCIÓN A LA LIBRERÍA PANDAS





INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Web:
  - https://pandas.pydata.org/
- Instalación:
  - https://pandas.pydata.org/docs/getting_started/index.html#getting-started

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- ¿Qué es pandas?
  - Es una biblioteca de Python para análisis y manipulación de datos.
  - Es muy útil en ciencia de datos por su equilibrio entre simplicidad y potencia.
  - Importación:
    - import pandas as pd
  - Estructuras de datos:
    - Serie (lista de datos con índices)
    - DataFrame (matriz de datos con índice)
  - Contenido de las estructuras de datos: índices (index) y columnas (columns)
    - Serie: Índices. Columnas.
    - DataFrame: Índices. Columnas.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Tipos de datos

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Tipos de datos:

Tipo pandas	Descripción
int64	Enteros
float64	Decimales
bool	Booleanos
string	Texto moderno
object	Texto/genérico
datetime64[ns]	Fechas
category	Categórico
Int64	Entero nullable

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Consulta de tipos:
  - df.dtypes → Tipos de las columnas →
  - df.info()

```
nombre_de_la_persona      str
edad_de_la_persona      int64
ciudad_de_la_persona      str
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   nombre_de_la_persona   3 non-null      str
 1   edad_de_la_persona     3 non-null      int64
 2   ciudad_de_la_persona   3 non-null      str
dtypes: int64(1), str(2)
memory usage: 204.0 bytes
None
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Conversiones:
  - Método **astype**:

```
df["edad"] = df["edad"].astype(int)
df["precio"] = df["precio"].astype(float)
df["activo"] = df["activo"].astype(bool)
```

```
df = df.astype({
    "edad": "int64",
    "precio": "float64",
    "nombre": "string"
})
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Conversiones:
  - Método **tonumeric**:

```
pd.to_numeric(df["columna"])
```

  - Método **tonumeric** con manejo de **errores:**

```
pd.to_numeric(df["columna"], errors="coerce")
```

  - Opciones de errors:
    - "raise" → lanza error
    - "coerce" → convierte inválidos en **NaN (Not a Number)**
    - "ignore" → deja valores originales

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Conversiones:
  - Método **to_datetime**:

```
df["fecha"] = pd.to_datetime(df["fecha"])
```

  - Método **to_datetime** con **formato:**

```
df["fecha"] = pd.to_datetime(
    df["fecha"],
    format="%Y-%m-%d"
)
```

  - Opciones de errors:
    - "raise" → lanza error
    - "coerce" → convierte inválidos en **NaT (Not a Time)**
    - "ignore" → deja valores originales

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Conversiones:
  - Método **to_timedelta**:

```
# 1. Define the offset
five_days = pd.to_timedelta("5 days")

# 2. Pass dates as a LIST []
dates = pd.Series(['2024-01-01', '2024-05-02'])

# 3. Convert to datetime and add the timedelta
dates = pd.to_datetime(dates) + five_days

print(dates)
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Conversiones:
  - Método **convert_dtypes**:
  - Intenta inferir automáticamente los mejores tipos modernos de pandas.
    - **df = df.convert_dtypes()**
  - Muy útil para:
    - String
    - Int64
    - Boolean
    - tipos nullable

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Conversiones:
  - Método **infer_objects**:
  - Intenta convertir columnas object a tipos más específicos.
    - **df = df.infer_objects()**
  - Más limitado que convert_dtypes().

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Conversiones:
  - Aplicando funciones nativas de Python con el método **apply**.

```
df["edad"] = df["edad"].apply(int)
df["precio"] = df["precio"].apply(float)
```

  - Utilizando el método **map**.

```
df["activo"] = df["activo"].map(bool)
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Tipo str

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - Se invocan sobre la propiedad **str** de los objetos **Serie** y de los objetos **Index**
  - Pueden requerir la invocación previa al método **astype**("string") para convertir los campos.

```
df['nombre'] = df['nombre'].str.upper()
```

```
df.index = ('ana','luis','mart@')
df.index.str.contains('@')
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Cambio de formato.**
    - lower() → pasa a minúsculas.
    - upper() → pasa a mayúsculas.
    - title() → capitaliza palabras.
    - capitalize() → capitaliza la primera
    - letra.swapcase() → invierte mayúsculas/minúsculas.
    - casefold() → normalización para comparación

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Limpieza de espacios.**
    - strip() → quita espacios al inicio y final.
    - lstrip() → quita espacios a la izquierda.
    - rstrip() → quita espacios a la derecha

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Búsqueda y validación.**
    - contains() → contiene patrón (incluye regex).
    - startswith() → empieza con (no incluye regex).
    - endswith() → termina con (no incluye regex).
    - match() → coincide desde el inicio (incluye regex).
    - fullmatch() → coincide toda la cadena (incluye regex)

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Sustitución y modificación.**
    - replace() → reemplaza texto o regex
    - repeat() → repite la cadena.
    - pad() → rellena con caracteres.
    - zfill() → rellena con ceros)

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **División y unión.**
    - split() → divide en partes.
    - rsplit() → divide desde la derecha
    - join() → une elementos

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Extracción (regex).**
    - extract() → extrae coincidencias de una cadena utilizando regex.
    - extractall() → extrae todas las coincidencias.
    - findall() → lista coincidencias regex

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Posición y conteo.**
    - find() → primera posición.
    - rfind() → última posición.
    - index() → como find, pero falla si no encuentra.
    - count() → cuenta apariciones.
    - len() → longitud

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Selección de partes.**
    - slice() → slicing.
    - slice_replace() → reemplaza un slice.
    - get() → obtiene elemento por posición.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Codificación.**
    - encode(). Convertir texto (Unicode) a bytes.
    - decode(). Convertir bytes a texto (Unicode).

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Comprobaciones de tipo/contenido.**
    - isalpha().
    - isalnum().
    - isdigit().
    - isspace().
    - islower().
    - isupper().
    - istitle().
    - isnumeric().
    - isdecimal().

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Métodos de string:
  - **Normalización y utilidades.**
    - .normalize() → normaliza Unicode.
    - translate() → traducción de caracteres.
    - wrap() → ajusta ancho de texto

```
# 1. Crear el DataFrame de ejemplo
data = {'periodo': ['T1', 'T2', 'T3', 'T4', 'TA'],
        'ventas' : [2000, 3000, 2500, 4000, 11500]}
df = pd.DataFrame(data)

# 2. Crear la tabla de traducción usando str.maketrans
tabla = str.maketrans({'T': 't', 'A': '@'})

# 3. Aplicar .str.translate
df['periodo'] = df['periodo'].str.translate(tabla)

print(df)
```

```
DataFrame Traducido:
  periodo  ventas
0      t1    2000
1      t2    3000
2      t3    2500
3      t4    4000
4      t@   11500
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Manejo de valores faltantes

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Manejo de valores faltantes:

Método / Función	Descripción	Ejemplo
isna()	Detecta valores faltantes (NaN, None)	df.isna()
isnull()	Alias de isna()	df.isnull()
notna()	Detecta valores no nulos	df.notna()
notnull()	Alias de notna()	df.notnull()
dropna()	Elimina filas o columnas con nulos	df.dropna()
fillna()	Rellena valores faltantes	df.fillna(0)
ffill()	Propaga el último valor válido hacia adelante	df.ffill()
bfill()	Propaga el siguiente valor válido hacia atrás	df.bfill()
interpolate()	Interpola valores faltantes	df.interpolate()
replace()	Reemplaza valores específicos por otros	df.replace(np.nan, 0)

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Manejo de valores faltantes:

Método / Función	Descripción	Ejemplo
combine_first()	Completa nulos usando otro DataFrame/Series	df1.combine_first(df2)
where()	Mantiene valores según condición y reemplaza el resto	df.where(cond)
mask()	Reemplaza valores que cumplen una condición	df.mask(cond)
drop_duplicates()	Puede ayudar a limpiar registros incompletos repetidos	df.drop_duplicates()
astype()	Requiere manejar nulos antes de convertir tipos clásicos	df["x"].fillna(0).astype(int)
convert_dtypes()	Usa tipos nullable modernos compatibles con NA	df.convert_dtypes()
pd.NA	Valor nulo escalar moderno de pandas	pd.NA
np.nan	Valor nulo numérico de NumPy	np.nan
None	Valor nulo nativo de Python	None
info()	Muestra cantidad de valores no nulos por columna	df.info()
value_counts(dropna=False)	Incluye nulos en conteo de valores	df["col"].value_counts(dropna=False)

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Parámero **inplace**

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Parámetro **inplace**

- En pandas, inplace=True se utiliza cuando quieres modificar directamente el DataFrame o Series original sin crear una nueva variable.

Caso	Ejemplo	Objetivo
Eliminar filas con nulos	df.dropna(inplace=True)	Limpiar datos rápidamente
Eliminar columnas	df.drop("col", axis=1, inplace=True)	Quitar columnas innecesarias
Rellenar valores faltantes	df.fillna(0, inplace=True)	Completar datos faltantes
Renombrar columnas	df.rename(columns={"a":"b"}, inplace=True)	Cambiar nombres directamente

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Parámetro **inplace**

Caso	Ejemplo	Objetivo
Ordenar datos	df.sort_values("edad", inplace=True)	Reordenar el DataFrame
Reemplazar valores	df.replace("N/A", 0, inplace=True)	Normalizar datos
Eliminar duplicados	df.drop_duplicates(inplace=True)	Limpiar registros repetidos
Cambiar índices	df.set_index("id", inplace=True)	Definir nuevo índice

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Series

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Series

```
import pandas as pd

serie = pd.Series(('Primavera','Verano','Otoño','Invierno'))

print(serie)
```



```
0     Primavera
1        Verano
2         Otoño
3      Invierno
dtype: str
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Series (con valores, índice, nombre y tipo implícito)

```
import pandas as pd
serie = pd.Series([8,20,12,-10], index=('Primavera','Verano','Otoño','Invierno'),
name='Facturación')
print(serie)
```



```
Primavera      8
Verano        20
Otoño         12
Invierno     -10
Name: Facturación, dtype:
int64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- Series (con valores, índice, nombre y **tipo explicito** → realiza conversión)

```
import pandas as pd
serie = pd.Series([8,20,12,-10], index=('Primavera','Verano','Otoño','Invierno'),
name='Facturación', dtype="float64")
print(serie)
```



```
Primavera      8.0
Verano        20.0
Otoño         12.0
Invierno     -10.0
Name: Facturación, dtype:
float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

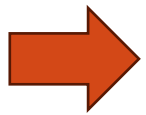
- Series → Tipos de datos

Pandas dtype	Descripción
object	Cadena de caracteres o mezcla de tipos
int64	Número entero
float64	Número decimal
bool	Valor booleano
datetime64	Fecha
timedelta	Intervalo de tiempo
category	Categorías

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Método **info** → Proporciona un resumen de la estructura de la serie:

```
import pandas as pd
a = pd.Series([8,20,12,-10,16], name='Diferencial')
print(a.info())
```



```
<class 'pandas.Series'>
RangeIndex: 5 entries, 0 to 4
Series name: Diferencial
Non-Null Count  Dtype
--------------  -----
5 non-null      int64
dtypes: int64(1)
memory usage: 172.0 bytes
None
```

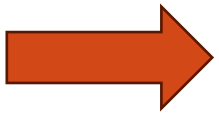
INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Método **describe** → Proporciona una serie con el resumen estadístico de variables numéricas:

```
import pandas as pd

a = pd.Series([8,20,12,-10,16])

print(a.describe())
print(type(a.describe()))
```

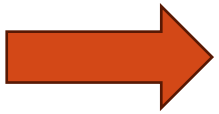


```
count    5.000000
mean     9.200000
std     11.627553
min    -10.000000
25%      8.000000
50%     12.000000
75%     16.000000
max     20.000000
dtype: float64
<class 'pandas.Series'>
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Método **head(n)** → Muestra primeras filas.
  - Método **tail(n)** → Muestra últimas filas.
  - Método **sample(n)** → Muestra filas aleatorias.
  - Indexadores:
    - loc → acceso por etiqueta → serie.loc[‘verano’]
    - iloc → acceso por índice → serie.iloc[0:-2]

```
import pandas as pd
a = pd.Series([8,20,12,-10],
index=('Primavera','Verano','Otoño','Invierno'),
name='Diferencial')
print(a.iloc[:-2])
print(a.loc['Verano'])
```



```
Primavera     8
Verano       20
Name: Diferencial, dtype: int64
20
```

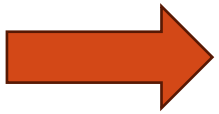
INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Asignación de índices explícitos mediante el atributo **index** en el constructor:
    - Los índices numéricos dejan de existir, cambiando acceso a la clave indicada.

```
import pandas as pd

a = pd.Series([1,2], index=['x','y'])
b = pd.Series([10,20], index=['y','z'])

print(a['x'])
print(b['z'])
```



```
1
20
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Acceso: serie[2]
  - Asignación: serie[2]='Autumn'

  - serie_1 = pd.Series([1,2,3])

  - Operaciones entre series y escalares:
    - +, -, *, /,**,%
    - Aplica la operación a cada elemento
    - Ejemplo: serie_1 ** 2

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones entre series:
    - +, -, *, /, **, %
    - Aplica la operación entre pares de los elementos de la serie.
    - Si hay distinto número de elementos → NaN
    - Ejemplo:

```
import pandas as pd

serie_1 = pd.Series([1,2,'a'])
serie_2 = pd.Series([2,2,'b','c'])

print(serie_1+serie_2)
```



```
0    3
1    4
2   ab
3  NaN
dtype: object
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

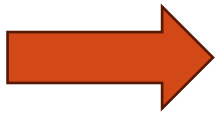
- **Series**
  - Método **add**:
    - Permite aplicar el operador + a los elementos de la serie.
    - El atributo fill_value permite indicar el valor de relleno ante series de distinto tamaño.

```
import pandas as pd

a = pd.Series([1,2])
b = pd.Series([10,20,8])

c = a.add(b, fill_value=1)

print(c)
```



```
0   11.0
1   22.0
2    9.0
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

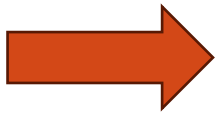
- **Series**
  - Método **sub**:
    - Permite aplicar el operador - a los elementos de la serie.
    - El atributo fill_value permite indicar el valor de relleno ante series de distinto tamaño.

```
import pandas as pd

a = pd.Series([1,2])
b = pd.Series([10,20,8])

c = a.sub(b, fill_value=2)

print(c)
```



```
0   -9.0
1  -18.0
2   -6.0
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Método **mul**:
    - Permite aplicar el operador * a los elementos de la serie.
    - El atributo fill_value permite indicar el valor de relleno ante series de distinto tamaño.

```
import pandas as pd

a = pd.Series([1,2])
b = pd.Series([10,20,8])

c = a.mul(b, fill_value=2)

print(c)
```



```
0    10.0
1    40.0
2    16.0
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

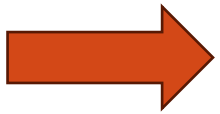
- **Series**
  - Método **div**:
    - Permite aplicar el operador / a los elementos de la serie.
    - El atributo fill_value permite indicar el valor de relleno ante series de distinto tamaño.

```
import pandas as pd

a = pd.Series([1,2])
b = pd.Series([10,20,8])

c = a.div(b, fill_value=2)

print(c)
```



```
0   0.10
1   0.10
2   0.25
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Método **pow**:
    - Permite aplicar el operador ** a los elementos de la serie.
    - El atributo fill_value permite indicar el valor de relleno ante series de distinto tamaño.

```
import pandas as pd

a = pd.Series([1,2])
b = pd.Series([10,20,8])

c = a.pow(b, fill_value=2)

print(c)
```



```
0   0.10
1   0.10
2   0.25
dtype: float64
```

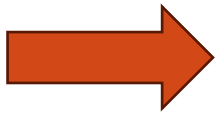
INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones de comparación:
    - <, >, <=, >=, ==, !=

```
import pandas as pd

a = pd.Series([8,20])
b = pd.Series([10,15])

print(a>b)
print(type(a>b))
```



```
0   False
1    True
dtype: bool
<class 'pandas.Series'>
```

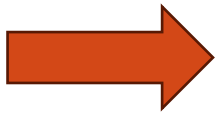
INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones de **filtro**:
    - Devuelven series con valores booleanos.
    - <, >, <=, >=, ==, !=

```
import pandas as pd

a = pd.Series([8,20,16])

print(a[a>10])
print(type(a[a>10]))
```



```
1   20
2   16
dtype: int64
<class 'pandas.Series'>
```

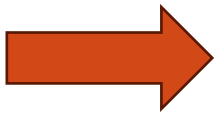
INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones **lógicas**:
    - Devuelven series con valores booleanos.
    - **& → and**
    - | → or
    - ~ → not

```
import pandas as pd

a = pd.Series([8,20,12,-10,16])

resultado = a[(a>10) & (a<18)]
print(resultado)
```



```
2   12
4   16
dtype: int64
```

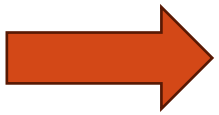
INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones **lógicas**:
    - Devuelven series con valores booleanos.
    - & → and
    - | → or
    - **~ → not**

```
import pandas as pd

a = pd.Series([8,20,12,-10,16])

resultado = a[~(a>15)]
print(resultado)
```



```
0    8
2   12
3  -10
dtype: int64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones **estadísticas**:
    - s.sum() → suma
    - s.mean() → media
    - s.median() → mediana
    - s.min() → mínimo
    - s.max() → máximo
    - s.std() → desviación estárndar
    - s.var() → varianza (muestral) → corrección de Bessel
    - s.count() → conteo

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones acumulativas:
  - s.cumsum() # suma acumulada
  - s.cumprod() # producto acumulado
  - s.cummax() # máximo acumulado (el máximo hasta el momento)
  - s.cummin() # mínimo acumulado (el mínimo hasta el momento)

```
import pandas as pd

a = pd.Series([8,20,12,-10,16])

print(a.cumsum())
print(type(a.cumsum()))
```



```
0    8
1   28
2   40
3   30
4   46
dtype: int64
<class 'pandas.Series'>
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones de ordenación y relacionadas con los índices:
  - s.sort_values() → Ordena por los valores
  - s.sort_index() → Ordena por los índices
  - s.reindex([2,1,0]) → Reasigna los índices

```
import pandas as pd

a = pd.Series([8,20,12])
print(a)
a=a.reindex([2,0,1])
print(a)
```



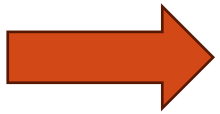
```
0    8
1   20
2   12
dtype: int64
2   12
0    8
1   20
dtype: int64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Funciones vectorizadas:
  - s.abs() → Obtiene el valor absolute.
  - s.round(*n*) → Redondea a *n* decimales.
  - s.clip(lower=0,upper=10) → Limita los valores mínimos y máximos, sustituyendo los valores por los umbrales indicados.

```
import pandas as pd

a = pd.Series([8.5,-20.1,12.9])
print(a.abs())
```



```
0    8.5
1   20.1
2   12.9
dtype: float64
```

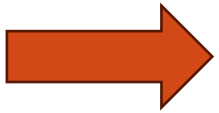
INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Aplicar funciones con el método **apply**:
    - s.apply(lambda x: x*2)

```
import pandas as pd

def doble(x):
    return 2*x

a = pd.Series([8,-20,12])
a = a.apply(doble)
print(a)
```



```
0   16
1  -40
2   24
dtype: int64
```

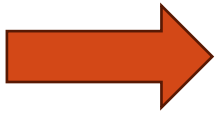
INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Asignación de nuevos valores mediante el método **map** (asignación mediante función):

```
import pandas as pd

def doble(x):
    return 2*x

a = pd.Series([8,-20,12])
a = a.map(func=doble)
# a = a.map(lambda x: 2*x)
print(a)
```



```
0    16
1   -40
2    24
dtype: int64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

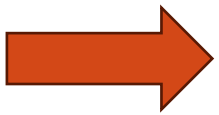
- **Series**
  - Asignación de nuevos valores mediante el método **map** (asignación explícita):

```
import pandas as pd

def doble(x):
    return 2*x

a = pd.Series([8,-20,12])
a = a.map({8:"A",12:"B", -20:"C"})

print(a)
```



```
0   A
1   C
2   B
dtype: str
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series. Manejo de valores faltantes:**
  - **s.isna() → Devuelve serie con valores True si falta el valor, False si hay valor.**
  - s.dropna()
  - s.fillna(0)

```
import pandas as pd

serie = pd.Series([8,-20,None,12])
print(serie.isna())
```



```
0    False
1    False
2     True
3    False
dtype: bool
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series. Manejo de valores faltantes:**
  - s.isna()
  - **s.dropna() → Elimina los valores faltantes**
  - s.fillna(0)

```
import pandas as pd

serie = pd.Series([8,-20,None,12])
print(serie.dropna())
```



```
0      8.0
1    -20.0
3     12.0
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series. Manejo de valores faltantes:**
  - s.isna()
  - s.dropna()
  - **s.fillna(*numero*) → Rellena los valores faltantes**

```
import pandas as pd

serie = pd.Series([8,-20,None,12])
print(serie.dropna())
```



```
0       8.0
1     -20.0
2     100.0
3      12.0
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series. Ejercicio:**

Fichero datos.txt:

```
10


30
20
80
```

Leer y reemplazar blanco por el promedio:

```
import pandas as pd

s = pd.read_csv("datos.txt", header=None, skip_blank_lines=False)[0]
media = s.mean()
s = s.fillna(media)
```

Obtiene la serie

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones con series de cadenas → Uso del elemento de acceso **str** que aplica el método siguiente a todos los elementos de la serie sin necesidad de bucles:

```
import pandas as pd

serie = pd.Series(['lunes', 'martes',
'miércoles', 'jueves', 'viernes'])
serie = serie.str.capitalize()
print(serie)
```



```
0         Lunes
1        Martes
2     Miércoles
3        Jueves
4       Viernes
dtype:  str
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones con series de cadenas → Uso del elemento de acceso **str** que aplica el método siguiente a todos los elementos de la serie sin necesidad de bucles.
  - Métodos:
    - serie.str.lower() # todo en minúsculas
    - serie.str.upper() # todo en mayúsculas
    - serie.str.capitalize() # primera letra mayúscula
    - serie.str.title() # tipo título
    - serie.str.strip() # quita espacios al inicio y final
    - serie.str.lstrip() # quita espacios a la izquierda
    - serie.str.rstrip() # quita espacios a la derecha
    - serie.str.contains("hola") # True/False si contiene "hola"
    - serie.str.startswith("a") # empieza por "a"
    - serie.str.endswith("z") # termina en "z"
    - serie.str.replace("a", "o") # reemplaza "a" por "o"
    - serie.str.split() # separa en lista de palabras
    - serie.str.split(",") # separa por coma
    - serie.str.get(0) # obtiene el primer elemento tras Split
    - serie.str.len() # longitud de cada string
    - serie.str.count("a") # cuenta cuántas "a" hay
    - serie.str.extract(r"(\d+)") # extrae números

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones con series de cadenas → Conversión a cadenas → Método de conversión **astype**

```
import pandas as pd


serie = pd.Series(['lunes', 'martes', 20,
'jueves', 'viernes'])
serie = serie.astype(str)
serie = serie.str.capitalize()
print(serie)
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones con series de fechas (**dt**):

```
import pandas as pd

serie = pd.Series(['03/09/1971', '31/12/2008'])
serie = pd.to_datetime(serie, dayfirst=True)
serie = serie.dt.year
print(serie)
```



```
0    1971
1    2008
dtype: int32
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones con series de fechas (**dt**):
  - Extraer componentes de fechas:
    - serie.dt.year # año
    - serie.dt.month # mes (1–12)
    - serie.dt.day # día
    - serie.dt.hour # hora
    - serie.dt.minute # minutos
    - serie.dt.second # segundos
    - serie.dt.weekday # día de la semana (0=lunes)
    - serie.dt.day_name() # nombre del día
    - serie.dt.month_name()# nombre del mes

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones con series de fechas (**dt**):
  - Redondeo de fechas:
    - serie.dt.round("H") # redondea a la hora
    - serie.dt.floor("D") # baja al día
    - serie.dt.ceil("D") # sube al día
  - Conversiones:
    - serie.dt.strftime("%Y-%m-%d") # a string con formato
  - Filtros:
    - serie[serie.dt.year == 2025]
    - serie[serie.dt.month == 1]
  - Zonas horarias:
    - serie.dt.tz_localize("UTC")
    - serie.dt.tz_convert("Europe/Madrid")

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Operaciones con series de fechas (**dt**):
  - Diferencias de tiempo (timedelta):
    - serie.dt.days # número de días
    - serie.dt.seconds # segundos (dentro del día, ojo con esto)
    - serie.dt.microseconds
    - serie.dt.nanoseconds

```
import pandas as pd

serie = pd.Series(['03/09/1971', '03/05/2026'])
serie = pd.to_datetime(serie, dayfirst=True)
diferencia = serie[1] - serie[0]
print(type(diferencia))
print(diferencia.days)
print(diferencia.days / 365.25)
```



```
<class 'pandas.Timedelta'>
19966
54.663928815879540
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Series**
  - Otras operaciones:
    - **diff() → obtiene la diferencia entre un elemento y su antecesor**
    - pct_change() → obtiene la diferencia porcentual entre un elemento y su antecesor
    - shift() → desplaza los datos hacia adelante.
    - rolling() → agrupa segmentos de datos

```
import pandas as pd


serie = pd.Series([3, 8, 14])
serie = serie.diff()
print(serie)
```



```
0     NaN
1     5.0
2     6.0
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Series

- Otras operaciones:
  - diff() → obtiene la diferencia entre un elemento y su antecesor
  - pct_change() → obtiene la diferencia porcentual entre un elemento y su antecesor
  - shift() → desplaza los datos hacia adelante.
  - **rolling() → agrupa segmentos de datos.**

```
import pandas as pd

serie = pd.Series([10, 20, 30, 40, 50, 60,
70, 80, 90, 100, 110, 120])
serie = serie.rolling(window=3).mean()
print(serie)
```



```
0      NaN
1      NaN
2     20.0
3     30.0
4     40.0
5     50.0
6     60.0
7     70.0
8     80.0
9     90.0
10   100.0
11   110.0
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Series

- Otras operaciones:
  - diff() → obtiene la diferencia entre un elemento y su antecesor
  - pct_change() → obtiene la diferencia porcentual entre un elemento y su antecesor
  - shift() → desplaza los datos hacia adelante.
  - **rolling() → agrupa segmentos de datos.**

```
import pandas as pd

serie = pd.Series([10, 20, 30, 40, 50, 60,
70, 80, 90, 100, 110, 120])
serie = serie.rolling(window=3).mean()
print(serie)
```



```
0      NaN
1      NaN
2     20.0
3     30.0
4     40.0
5     50.0
6     60.0
7     70.0
8     80.0
9     90.0
10   100.0
11   110.0
dtype: float64
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Dataframes

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes**
  - Un DataFrame es la estructura principal de la librería **pandas** para trabajar con datos tabulares en Python
  - Es, básicamente, una tabla bidimensional (filas y columnas) donde:
    - cada columna tiene un nombre y normalmente representa una variable (edad, nombre, salario)
    - cada fila representa un registro (una persona, una venta, un evento)
    - cada columna puede contener distintos tipos de datos (texto, números, fechas, etc.).

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes**
  - Creación:

```
import pandas as pd


df = pd.DataFrame({
    "nombre": ["Ana", "Luis", "Marta"],
    "edad": [25, 30, 28],
    "ciudad": ["Madrid", "Sevilla", "Valencia"]
})


print(df)
```



```
  nombre  edad    ciudad
0    Ana    25    Madrid
1   Luis    30   Sevilla
2  Marta    28  Valencia
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Carga de datos.**

Método	Descripción
pd.read_csv()	Carga datos desde archivos CSV (valores separados por comas u otros delimitadores).
pd.read_excel()	Carga datos desde archivos de Microsoft Excel (.xls, .xlsx).
pd.read_json()	Carga datos en formato JSON. Muy útil para APIs.
pd.read_sql()	Carga datos desde bases de datos usando consultas SQL.
pd.read_table()	Carga archivos de texto delimitados (por tabulaciones u otros separadores).
pd.read_parquet()	Carga archivos Parquet, formato optimizado para grandes volúmenes de datos.
pd.read_html()	Extrae tablas HTML de páginas web y las convierte en DataFrames.
pd.read_xml()	Carga datos estructurados en XML.
pd.read_pickle()	Carga objetos de pandas guardados en formato binario (pickle).
pd.DataFrame()	Crea un DataFrame manualmente a partir de listas, diccionarios u otras estructuras.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes**
  - **Atributos** informativos:

Atributo	Qué muestra
df.shape	Dimensiones del DataFrame (filas, columnas)
df.columns	Nombres de las columnas
df.index	Índices de las filas
df.dtypes	Tipo de dato de cada columna
df.size	Número total de elementos
df.ndim	Número de dimensiones (normalmente 2)
df.values	Datos en formato array de NumPy
df.empty	Indica si el DataFrame está vacío (True/False)

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes**
  - **Métodos** informativos:

Método	Qué muestra
df.head()	Primeras filas
df.tail()	Últimas filas
df.info()	Resumen general de estructura y tipos
df.describe()	Estadísticas descriptivas de columnas numéricas
df.sample()	Muestra filas aleatorias
df.nunique()	Número de valores únicos por columna
df.isnull()	Detecta valores nulos
df.duplicated()	Detecta filas duplicadas

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes**
  - Atributo **columns**:
    - Contiene los nombres de las columnas.
    - Se pueden (deben) modificar para facilitar su tratamiento posterior.

```
df.columns = df.columns.str.replace(' ','_').str.lower()
```

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes**
  - Atributo **index**:
    - Contiene los índices de las columnas.
    - Son conjuntos (admiten operaciones de conjuntos)
    - Se pueden modificar para facilitar su tratamiento posterior.
    - Se puede indicar que una columna es índice (método **set_index**).
    - Se puede resetear a índice numérico (método **reset_index**).
    - Se pueden renombrar con el método **rename**.
    - Pueden formar parte de los filtros.
    - Se pueden ordenar con el método **sort_index.**

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes**
  - Atributo **index. Propiedades:**
    - df.index.name
    - df.index.shape
    - df.index.dtype
    - df.index.size
    - df.index.is_unique
    - df.index.has_duplicates

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Inspección y exploración.**
  - head(): muestra las primeras filas.
  - tail(): muestra las últimas filas.
  - info(): resumen estructural (tipos, nulos, memoria).
  - describe(): estadísticas descriptivas.
  - shape: devuelve dimensiones (filas, columnas).
  - dtypes: muestra tipos de datos por columna.
  - columns: lista nombres de columnas.
  - index: devuelve etiquetas del índice.
  - sample(): muestra una muestra aleatoria.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Selección y acceso a datos.**
  - Selección de columnas ([]): acceder a columnas concretas.
  - loc[]: selección por etiquetas.
  - iloc[]: selección por posición.
  - at[]: acceso rápido a un único valor por etiqueta.
  - iat[]: acceso rápido a un único valor por posición.
  - Filtrado booleano: selección basada en condiciones.
  - query(): filtrado con expresiones.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Modificación y transformación.**
  - Asignación directa: crear o modificar columnas.
  - assign(): crear columnas de forma encadenable.
  - rename(): renombrar columnas o índices.
  - replace(): sustituir valores.
  - astype(): cambiar tipos de datos.
  - update(): actualizar valores con otro DataFrame.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Manejo de valores nulos.**
  - isna() / isnull(): detectar nulos.
  - notna() / notnull(): detectar valores válidos.
  - dropna(): eliminar filas o columnas con nulos.
  - fillna(): rellenar nulos.
  - interpolate(): interpolar valores faltantes.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Ordenación.**
  - sort_values(): ordenar por valores de columnas.
  - sort_index(): ordenar por índice.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Agregación y estadística.**
  - sum(): suma.
  - mean(): media.
  - median(): mediana.
  - min(): mínimo.
  - max(): máximo.
  - count(): cuenta valores no nulos.
  - std(): desviación estándar.
  - var(): varianza.
  - nunique(): cuenta valores únicos.
  - value_counts(): frecuencia de valores.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Agrupación.**
  - groupby(): agrupa por una o más columnas.
  - agg(): aplica varias agregaciones.
  - transform(): transforma manteniendo forma.
  - filter(): filtra grupos.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Aplicación de funciones.**
  - apply(): aplica funciones por filas o columnas.
  - applymap(): aplica función elemento a elemento (en versiones clásicas).
  - map() (Series): transformación elemento a elemento.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Combinación de DataFrames.**
  - merge(): combinación tipo SQL.
  - join(): unión basada en índices.
  - concat(): concatenación vertical u horizontal.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Reestructuración.**
  - melt(): convertir columnas en filas → **convierte formato ancho a largo**.
  - pivot(): reorganizar filas en columnas → **convierte formato largo a ancho**.
  - pivot_table(): reorganizar y agregar datos duplicados → **convierte formato largo a ancho**.
  - stack(): comprime columnas en índice.
  - unstack(): expande índices a columnas..

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Reestructuración.**

Método	Transformación	Uso
melt()	Wide → Long	Convertir columnas en filas
pivot()	Long → Wide	Reorganizar filas en columnas
pivot_table()	Long → Wide	Reorganizar y agregar datos duplicados
stack()	Columnas → niveles del índice	Trabajar con índices jerárquicos
unstack()	Niveles del índice → columnas	Expandir índices jerárquicos

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Reestructuración. Melt.**

```
df_wide = pd.DataFrame({
    "persona": ["Ana", "Luis"],
    "enero": [60, 75],
    "febrero": [61, 74],
    "marzo": [62, 73]
})
```

	persona	enero	febrero	marzo
0	Ana	60	61	62
1	Luis	75	74	73

```
df_long = df_wide.melt(
    id_vars="persona",
    var_name="mes",
    value_name="peso"
)
```

	persona	mes	peso
0	Ana	enero	60
1	Luis	enero	75
2	Ana	febrero	61
3	Luis	febrero	74
4	Ana	marzo	62
5	Luis	marzo	73

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Índices.**
  - set_index(): define columnas como índice.
  - reset_index(): restaura índice normal.
  - reindex(): alinea con nuevo índice.
  - rename_axis(): renombra ejes.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Eliminación de datos.**
  - drop(): elimina filas o columnas.
  - pop(): extrae y elimina una columna.
  - drop_duplicates(): elimina duplicados.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Valores únicos o duplicados.**
  - unique() (Series): valores únicos.
  - duplicated(): marca duplicados.
  - drop_duplicates(): elimina duplicados.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Operaciones temporales.**
  - resample(): reagrupar por frecuencia temporal.
  - shift(): desplazar valores.
  - rolling(): ventanas móviles.
  - expanding(): acumulados progresivos.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Entradas y salidas.**
  - read_csv() / to_csv()
  - read_excel() / to_excel()
  - read_json() / to_json()
  - read_sql() / to_sql()
  - to_parquet()to_pickle()

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Comparación.**
  - equals(): compara igualdad exacta.
  - compare(): muestra diferencias.
  - isin(): verifica pertenencia.

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Dataframes. Copia y gestión de memoria.**
  - copy(): crea copia explícita.
  - memory_usage(): consumo de memoria..

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Visualización

INTRODUCCIÓN A LA LIBRERÍA PANDAS

Chart visualization.

- https://pandas.pydata.org/docs/user_guide/visualization.html

- plot.line() # Gráfico de líneas
- plot.bar() # Gráfico de barras verticales
- plot.barh() # Gráfico de barras horizontales
- plot.hist() # Histograma
- plot.box() # Diagrama de caja y bigotes (boxplot)
- boxplot() # Otra forma de crear boxplots
- plot.kde() # Curva de densidad KDE
- plot.density() # Igual que kde(), densidad
- plot.area() # Gráfico de áreas
- plot.pie() # Gráfico circular / de pastel
- plot.scatter() # Diagrama de dispersión

INTRODUCCIÓN A LA LIBRERÍA PANDAS

- **Table Visualization.**
  - https://pandas.pydata.org/docs/user_guide/style.html

	A	B	C	D
0	0.125730	-0.132105	0.640423	0.104900
1	-0.535669	0.361595	1.304000	0.947081
2	-0.703735	-1.265421	-0.623274	0.041326
3	-2.325031	-0.218792	-1.245911	-0.732267
4	-0.544259	-0.316300	0.411631	1.042513
5	-0.128535	1.366463	-0.665195	0.351510
6	0.903470	0.094012	-0.743499	-0.921725
7	-0.457726	0.220195	-1.009618	
8	-0.159225	0.540846	0.214659	
9	-0.653829	-0.129614	0.783975	

	A	B	C	D
0	0.125730	-0.132105	nan	0.104900
1	-0.535669	0.361595	1.304000	0.947081
2	-0.703735	-1.265421	-0.623274	0.041326
3	-2.325031	-0.218792	-1.245911	-0.732267
4	-0.544259	-0.316300	0.411631	
5	-0.128535	1.366463	-0.665195	
6	0.903470	0.094012	-0.743499	
7	-0.457726	0.220195	-1.009618	
8	-0.159225	0.540846	0.214659	
9	-0.653829	-0.129614	0.783975	

	A	B	C
0	0.125730	-0.132105	nan
1	-0.535669	0.361595	1.304000
2	-0.703735	-1.265421	-0.623274
3	-2.325031	-0.218792	-1.245911
4	-0.544259	-0.316300	0.411631

	0	1	2	3	4
0	0.9	-0.7	0.7	-0.4	-1.6
1	-0.0	0.6	1.1	0.0	-1.1
2	-1.1	-1.1	-1.8	0.6	-1.4
3	-1.7	-1.3	0.1	0.3	1.2
4	0.9	-0.5	0.9	-1.1	0.2
Sum	-1.094	-3.052	0.980	-0.541	-2.703
Average	-0.219	-0.610	0.196	-0.108	-0.541

Weather Conditions

	Tokyo	Beijing
Monday	Heavy Rain	Heavy Rain
Tuesday	Heavy Rain	Dry
Wednesday	Heavy Rain	Dry
Thursday	Heavy Rain	Heavy Rain
Friday	Heavy Rain	Heavy Rain